CW1 #9.1: `LabeledCircle` Class with Exceptions

Improve the `LabeledCircle` class, the subclass of `Circle` last week (Week 8) to also handle exceptional behaviors using exceptions. The new requirements will be given in *blue italic*.

Note that an implementation of `Circle` (from the Week 6 CW1) has been provided in LMO.

Requirements

1. **Class Declaration**
   - `public class LabeledCircle extends Circle`

2. **State (new fields only)**
   - `private String label` — short text to identify the circle.
   - `private boolean filled` — whether the circle is visually filled (`true`) or just an outline (`false`).

   Keep centers and radius in the **superclass** (as they already exist in `Circle`).

3. **Constructors:** Provide two constructors that delegate appropriately to `Circle` via `super(...)`:
   - A **no-argument constructor** that uses super() to create a circle centred at the origin `(0, 0)` with radius `1.0`, then sets `label` to `"untitled"` and `filled` to `false`.
   - A **full constructor:** `public LabeledCircle(double centerX, double centerY, double radius, String label, boolean filled)`:
     - Must call `super(centerX, centerY, radius)`.
     - Sets the new fields accordingly (if `label` is null, store `"untitled"`).
   - *If the radius is zero, throw an `IllegalArgumentException` with the error message `"Radius cannot be zero!"`.*
   - *If the radius is negative, throw an `IllegalArgumentException` with the error message `"Radius cannot be negative!"`.*

4. **Getters & Setters (for new fields)**
   - `public String getLabel()`
   - `public boolean isFilled()`
   - `public void setLabel(String newLabel)` — if `newLabel` is `null`, store `"untitled"`.
   - `public void setFilled(boolean filled)`

5. **New Behaviours (added functionality):** Implement both of the following methods using the public methods of `Circle` (e.g., its getters and `setCenter`):
   - `public double getDiameter()`: Returns `2.0 * getRadius()`.

- `public void moveBy(double dx, double dy)`: Translates the circle by `(dx, dy)`. Hint: use `getCenterX()`, `getCenterY()`, and `setCenter(...)` in `Circle`.
- *`public double getDiameterDividedBy(double divisor)`: it returns the diameter divided by `divisor`, but throws an `ArithmeticException` with the error message `"Don't divide by zero!"`.*
- *`public double getDiameterDividedBy(double divisor, String errorMessage)`: it returns the diameter divided by `divisor`, but throws an `ArithmeticException` with the error message `errorMessage`.*
- *`public double setLabelDiameterDividedBy(double divisor, String newLabel)`: it calls and returns the result the `getDiameterDividedBy(double)` method, but if it detects that an `ArithmeticException` object has been thrown, it will catch it and print its error message, as well as set the label of the circle to `newLabel`, and return 0.0.*

6. **Overriding:** Override `toString()` from `Circle`:
   - Return a string like: `LabeledCircle(center=(2.0, -1.5), radius=3.0, label="Room A", filled=true)`
   - **Hint:** You can add a double quotation mark character to a string like this: `String s = "\"";`

7. ***Overriding:*** *Override `setRadius(double newRadius)` from `Circle`:*
   - *If the radius is zero, throw an `IllegalArgumentException` with the error message `"Radius cannot be zero!"`.*
   - *If the radius is negative, throw an `IllegalArgumentException` with the error message `"Radius cannot be negative!"`.*
   - *Otherwise set the radius to be `newRadius`.*

Notes:

- **Do not** copy or modify `Circle.java`. Your code must work with the given `Circle` class from the Week 6 CW1.
- Keep fields in `LabeledCircle` **private** and use getters/setters.
- Use the given skeleton class code `LabeledCircle.java` file, but only submit the constructors and the methods to Learning Mall.

Example Test Cases

```
// Example 1: creating new LabeledCircle with zero radius
try {
    LabeledCircle a = new LabeledCircle(1.0, 2.0, 0.0, "A", true);
} catch (IllegalArgumentException iae) {
    System.out.println(iae.getMessage()); // -> Radius cannot be zero!
}
```

```java
// Example 2: getDiameterDividedBy zero
LabeledCircle b = new LabeledCircle(1.0, 2.0, 3.0, "B", false);
try {
    double result = b.getDiameterDividedBy(0.0);
} catch (ArithmeticException ae) {
    System.out.println(ae.getMessage()); // -> Don't divide by zero!
}


// Example 3: setRadius is given zero
LabeledCircle c = new LabeledCircle(1.0, 2.0, 3.0, "C", true);
try {
     c.setRadius(0.0);
} catch (IllegalArgumentException iae) {
     System.out.println(iae.getMessage()); // -> Radius cannot be zero!
}


// Create more test cases to intensively evaluate your implementation, as
there will be more hidden test cases.
```

Submission guidance

- Submit only the content of your `LabeledCircle.java` file.
- Include all constructors, getters, setters, and methods exactly as listed in Learning Mall Quiz.